

How to Use This Coding Guide

The technical round includes core Python problems - the classic 'Happy Number' question was explicitly reported.

These cover number theory, string handling and common built-in logic expected of a data engineer.

Every Python solution has been executed and verified.

Coding Problems

Problem 1: Happy Number

Difficulty: *Easy* | Pattern: *Cycle Detection (Hash Set)*

Problem Statement

A happy number is defined by repeatedly replacing the number with the sum of the squares of its digits; if this process reaches 1 the number is happy, otherwise it loops endlessly. Determine whether n is a happy number.

Input Format

A single integer n .

Output Format

'True' if n is happy, otherwise 'False'.

Constraints

$1 \leq n \leq 2^{31} - 1$

Sample Input

19

Sample Output

True

Python Solution

```
def is_happy(n):
    seen = set()
    while n != 1 and n not in seen:
        seen.add(n)
        n = sum(int(d) ** 2 for d in str(n))
    return n == 1

print(is_happy(19))
```

Step-by-Step Explanation

Repeatedly transform n into the sum of the squares of its digits. Track seen values to detect an endless loop. If we reach 1 the number is happy. $19 \rightarrow 82 \rightarrow 68 \rightarrow 100 \rightarrow 1$.

Dry Run

```
19 -> 1^2+9^2 = 82
82 -> 64+4 = 68
68 -> 36+64 = 100
100 -> 1 -> stop, n==1 -> True
```

Time Complexity: **$O(\log n)$ per step** | Space Complexity: **$O(\log n)$**

Problem 2: Factorial of a Number

Difficulty: **Easy** | Pattern: **Iteration / Recursion**

Problem Statement

Compute the factorial $n!$ of a non-negative integer n .

Input Format

A single integer n .

Output Format

The value of $n!$.

Constraints

$0 \leq n \leq 20$ (fits in 64-bit)

Sample Input

5

Sample Output

120

Python Solution

```
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

print(factorial(5))
```

Step-by-Step Explanation

Multiply the integers from 2 up to n . By convention $0!$ and $1!$ equal 1, which the loop handles because it does not execute. $5! = 1*2*3*4*5 = 120$.

Dry Run

```
result=1
*2=2
*3=6
*4=24
*5=120
Result = 120
```

Time Complexity: **$O(N)$** | Space Complexity: **$O(1)$**

Problem 3: Check Prime Number

Difficulty: **Easy** | Pattern: **Trial Division up to $\sqrt{n}$**

Problem Statement

Determine whether a given integer n is a prime number.

Input Format

A single integer n .

Output Format

'True' if prime, otherwise 'False'.

Constraints

$1 \leq n \leq 10^9$

Sample Input

29

Sample Output

True

Python Solution

```
def is_prime(n):
    if n < 2:
        return False
    i = 2
    while i * i <= n:
        if n % i == 0:
            return False
        i += 1
    return True

print(is_prime(29))
```

Step-by-Step Explanation

A number is prime if it has no divisor between 2 and its square root. Checking only up to $\sqrt{n}$ is enough because a larger factor would pair with a smaller one. 29 has no such divisor.

Dry Run

```
n=29
i=2 29%2!=0
i=3 no
i=4 no
i=5 (25<=29) no
i=6 (36>29) stop -> True
```

Time Complexity: $O(\sqrt{N})$ | Space Complexity: $O(1)$

Problem 4: Palindrome Number

Difficulty: *Easy* | Pattern: *Digit Reversal*

Problem Statement

Determine whether an integer reads the same forwards and backwards. Negative numbers are not palindromes.

Input Format

A single integer n.

Output Format

'True' or 'False'.

Constraints

$-2^{31} \leq n \leq 2^{31} - 1$

Sample Input

121

Sample Output

True

Python Solution

```
def is_palindrome(n):
    if n < 0:
        return False
    original, reversed_num = n, 0
    while n > 0:
        reversed_num = reversed_num * 10 + n % 10
        n //= 10
    return original == reversed_num

print(is_palindrome(121))
```

Step-by-Step Explanation

Reverse the digits mathematically by repeatedly taking the last digit ($n \% 10$) and building the reversed number. Compare with the original. 121 reversed is 121.

Dry Run

```
n=121 rev=0
rev=1 n=12
rev=12 n=1
rev=121 n=0
121==121 -> True
```

Time Complexity: **$O(\log n)$** | Space Complexity: **$O(1)$**

Problem 5: Greatest Common Divisor (GCD)

Difficulty: *Easy* | Pattern: *Euclidean Algorithm*

Problem Statement

Compute the greatest common divisor of two positive integers.

Input Format

Two integers a and b.

Output Format

The GCD of a and b.

Constraints

$1 \leq a, b \leq 10^9$

Sample Input

48 18

Sample Output

6

Python Solution

```
def gcd(a, b):  
    while b:  
        a, b = b, a % b  
    return a  
  
print(gcd(48, 18))
```

Step-by-Step Explanation

The Euclidean algorithm repeatedly replaces (a, b) with (b, a mod b) until b is zero; a is then the GCD.
gcd(48,18) -> gcd(18,12) -> gcd(12,6) -> gcd(6,0) = 6.

Dry Run

```
a=48 b=18 -> a=18 b=12  
-> a=12 b=6  
-> a=6 b=0  
b==0 -> return 6
```

Time Complexity: $O(\log(\min(a, b)))$ | Space Complexity: $O(1)$

Problem 6: Count Vowels in a String

Difficulty: *Easy* | Pattern: *String Traversal*

Problem Statement

Count how many vowels (a, e, i, o, u, case-insensitive) appear in a given string.

Input Format

A single line: the string.

Output Format

The number of vowels.

Constraints

$1 \leq \text{length} \leq 10^5$

Sample Input

Data Engineering

Sample Output

7

Python Solution

```
def count_vowels(text):  
    vowels = set('aeiou')  
    return sum(1 for ch in text.lower() if ch in vowels)  
  
print(count_vowels('Data Engineering'))
```

Step-by-Step Explanation

Lowercase the string and count characters that belong to the vowel set. 'Data Engineering' contains a, a, e, i, e, e, i = 7 vowels.

Dry Run

```
Data -> a,a (2)
Engineering -> e,i,e,e,i (5)
Total = 7
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

BI3 Technologies

Python Fundamentals & Problems

Applied Role(s):

Data Engineer - Trainee

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

Table of Contents

How to Use This Coding Guide	1
Coding Problems	1